

CS 25000 – Labs 07 & 08 Review and Correct Answers (English)

Lab 07 – Generating Executables

This lab explains the stages that convert a C program into an executable: preprocessing, compiling, assembling, and linking.

- **Preprocessor:** Removes `#define`, `#include`, and comments. It replaces constants and macros to save run-time by doing computations at compile-time.
 - **Compiler:** When compiling `hello.i` directly, errors occur because of stray characters. After preprocessing correctly, `gcc -S hello.i -o hello.s` produces assembly for AArch64 (by Arm Holdings).
 - **Assembler:** Converts `hello.s` into `hello.o`. Running `nm hello.o` shows `T main` (defined) and `U printf, puts` (undefined).
 - **Linker:** Combines object files into an executable. `main` is placed at address `0x7d8` (2008 bytes before start). Shared library functions (`printf`, `puts`) are resolved at load time.
 - **Optimization:** `gcc` replaces `printf()` with `puts()` when possible for efficiency.
- Conclusion:** Lab 07 demonstrates the full path from source code to binary executable and the roles of each stage.

Lab 08 – Data Dependence

This lab covers instruction dependencies and LEGv8 register conventions.

1. **Pipelined Processors:** Allow multiple instructions in flight, increasing throughput. Software developers benefit indirectly from speed.
 2. **Great Idea:** “Make the common case fast” — provide enough registers for passing arguments.
 3. **Registers not preserved:** `X0–X15` are caller-saved; used for arguments and temporaries.
 4. **Using X20:** Must save/restore around the call: `STUR X20, [SP, #-16]!`
`LDUR X20, [SP], #16`
 5. **SUB dependence example:** `SUB X9, X7, X8`
`SUB X8, X9, X5`
 6. **Dependence tuples solution:** `ADD X4, X1, X2`
`AND X6, X1, X3`
`SUB X5, X4, X6`
`XOR X1, X6, X4`
 7. **Adding NOPs:** `ADD X1, X2, X3`
`NOP`
`AND X4, X1, X6`
`NOP`
`SUB X5, X1, X8`
 8. **Reordering:** Not possible without breaking RAW dependences on `X1`.
 9. **Runtime true dependence uncertain:** `LDUR X1, [X2]`
`STUR X3, [X2]`
 10. **Removable dependences:** WAR and WAW can be eliminated by register renaming; RAW cannot.
- Conclusion:** Understanding data dependences helps improve pipeline efficiency and avoid stalls.